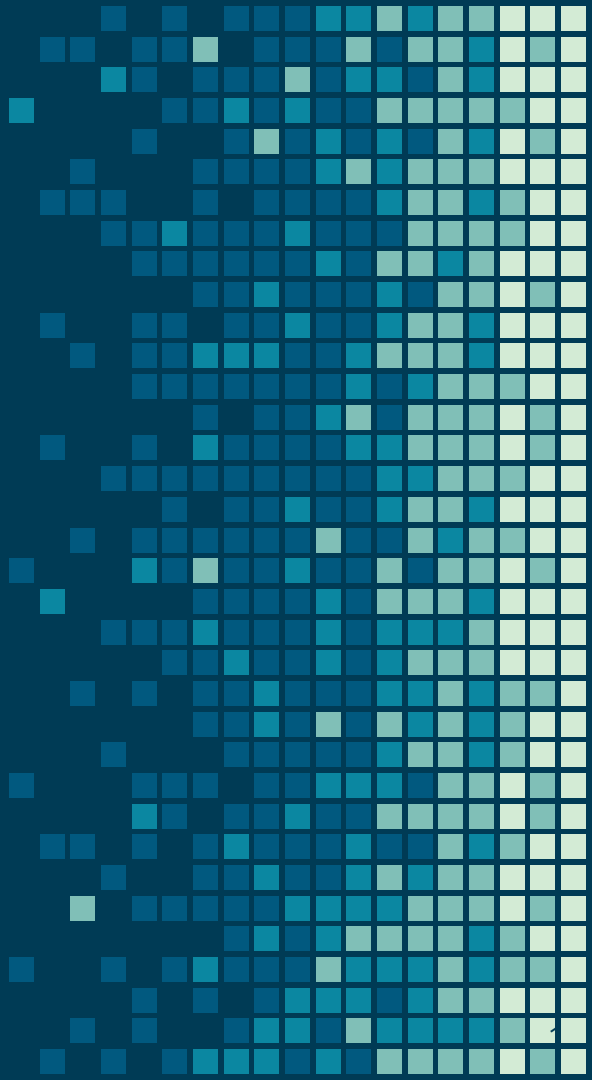
EMBEDDED CONTROL

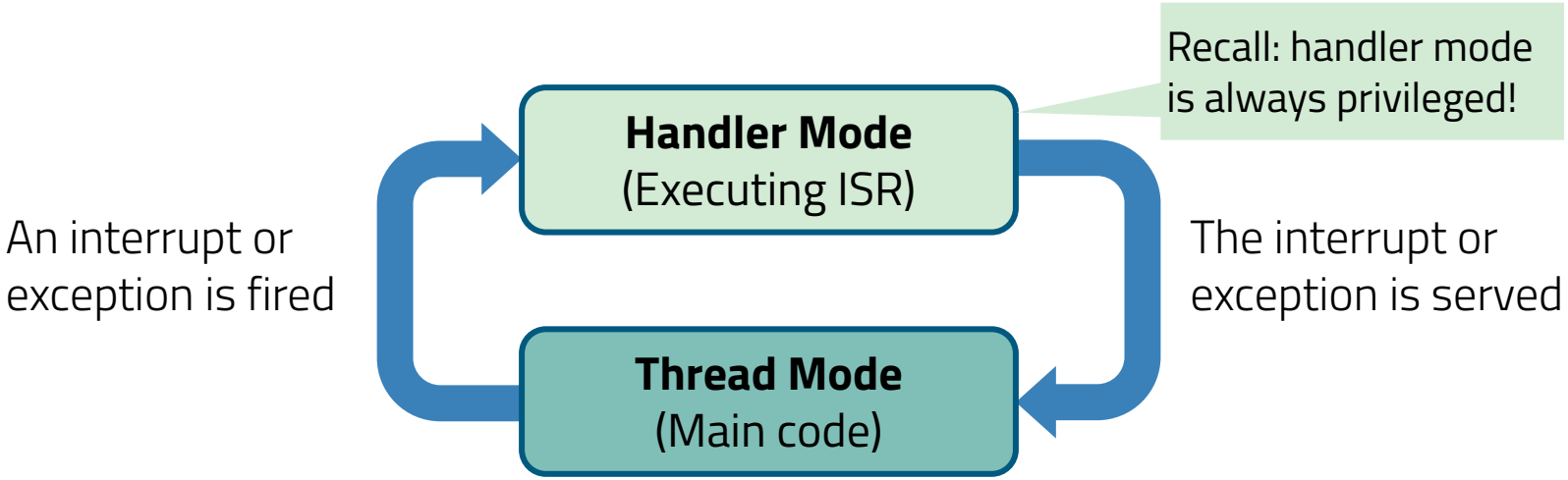
Prof. Luca De Cicco
luca.decicco@poliba.it
https://c3lab.poliba.it/LDC



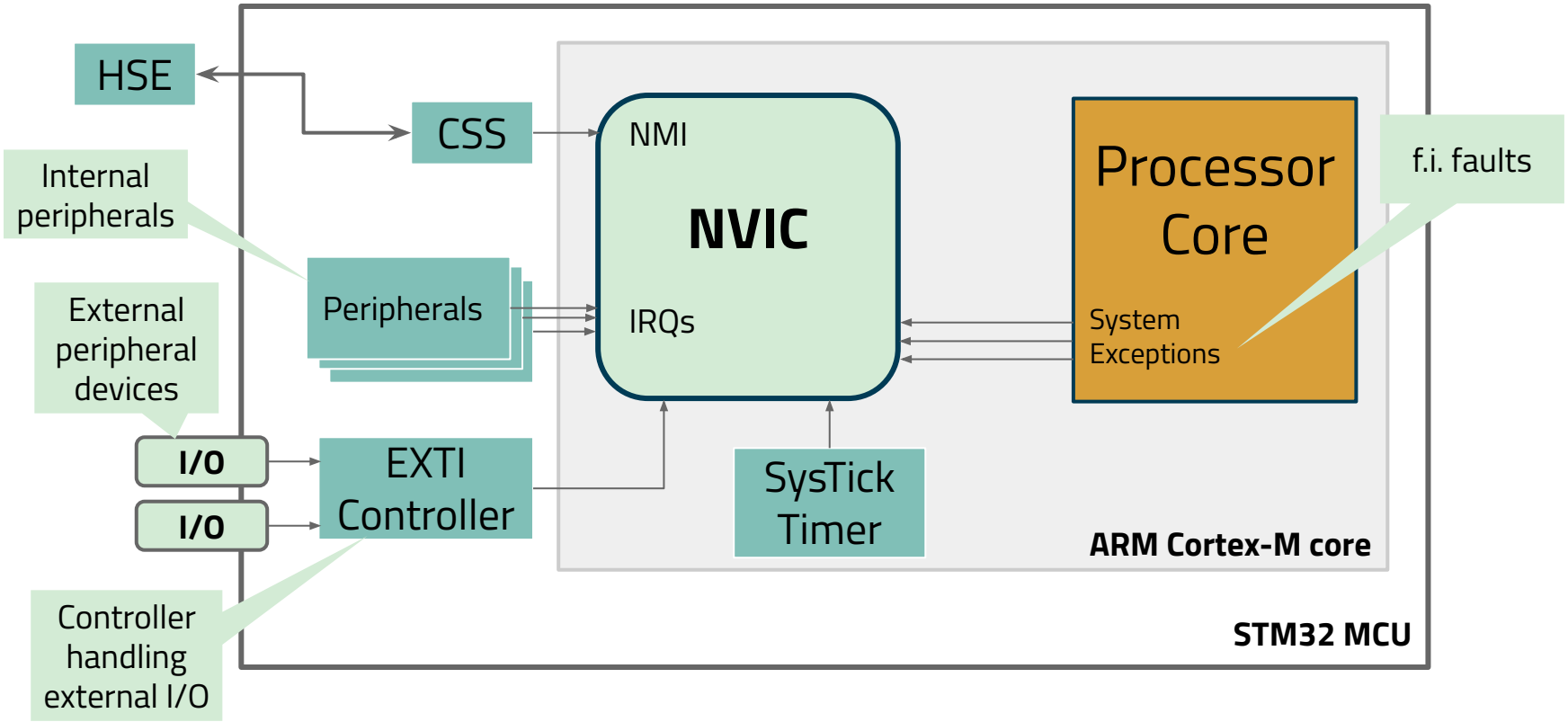
ARM Exception handling

How does it work in ARM-based MCUs?

- Back to lecture #3 (ARM programming model)
- **Thread mode**: normal mode of operation (main code)
- **Handler mode**: executing code to *serve* an interrupt (ISR)



Nested Vectored Interrupt Controller (NVIC)



Nested Vectored Interrupt Controller (NVIC)

- The NVIC is a hardware programmable through registers used by **all Cortex-M microcontrollers** that handles exceptions
- It handles exceptions generated:
  - **Internally to the Cortex-M Core**:
    - Processor core: System exceptions
    - SysTick timer: periodical interrupt typically used for RTOS schedulers
  - **External of the Cortex-M core but inside the MCU**:
    - Peripherals: internal peripherals generate interrupts
    - Clock security system (CSS): fires an interrupt when external clock (HSE) failure is detected
  - **External of the MCU**: external I/O HW connected to the EXTI (External Input Event Controller) controller in the MCU

Cortex-M3/4/7 Vector Table

- Each exception is associated to an **exception number**
- Cortex-M processors support up to 256 types of exceptions numbered from **-15 to 240** in CMSIS-Core
- Exception numbers are divided into two groups:
  - First 16 are for **system exceptions**
  - Remaining 240 are for **interrupt requests (IRQ)**
- CMSIS-Core allocates to **systems exceptions** numbers from **-14 to -1**, and to IRQs numbers from **0 to 239**
- CMSIS-Core also standardizes the exception handler function names for all system exceptions

CMSIS-Core Exception Definitions

System Exceptions

Ex. num	Type	CMSIS-Core Value	Exc. hanlder name
1	**Reset**	not specified	`Reset_Handler`
2	NMI	-14	`NMI_Handler`
3	Hard Fault	-13	`HardFault_Handler`
4	MemManage Fault	-12	`MemManage_Handler`
5	Bus Fault	-11	`BusFault_Handler`
6	Usage Fault	-10	`UsageFault_Handler`
7...10	RESERVED		
11	SVC	-5	`SVC_Handler`
12	Debug Monitor	-4	`DebugMon_Handler`
13	RESERVED		
14	PendSV	-2	`PendSV_Handler`
15	SYSTICK	-1	`SysTick_Handler`
16-...256	Interrupt #0-#239	**0 to 239**	*`device specific`*

Peripherals exceptions

Example: STM32F446XX vector table

_estack = end of SRAM (start address of stack) defined in `stm32_flash.ld`

`startup_stm32f446xx.s`

```
144 g_pfnVectors:
145   .word   _estack
146   .word   Reset_Handler
147   .word   NMI_Handler
148   .word   HardFault_Handler
149   .word   MemManage_Handler
150   .word   BusFault_Handler
151   .word   UsageFault_Handler
152   .word   0
153   .word   0
154   .word   0
155   .word   0
156   .word   SVC_Handler
157   .word   DebugMon_Handler
158   .word   0
159   .word   PendSV_Handler
160   .word   SysTick_Handler
```

Highest priority interrupt called every time a reset occurs

`0` means "reserved"

System Exceptions

Default ISR names defined by ST

```
162  /* External Interrupts */
163  .word     WWDG_IRQHandler
164  .word     PVD_IRQHandler
165  .word     TAMP_STAMP_IRQHandler
166  .word     RTC_WKUP_IRQHandler
167  .word     FLASH_IRQHandler
168  .word     RCC_IRQHandler
169  .word     EXTI0_IRQHandler
170  .word     EXTI1_IRQHandler
171  .word     EXTI2_IRQHandler
172  .word     EXTI3_IRQHandler
173  .word     EXTI4_IRQHandler
174  .word     DMA1_Stream0_IRQHandler
     ...       ...
250  .word     SAI1_IRQHandler
251  .word     0
252  .word     0
253  .word     0
254  .word     SAI2_IRQHandler
255  .word     QuadSPI_IRQHandler
256  .word     CEC_IRQHandler
257  .word     SPDIF_RX_IRQHandler
258  .word     FMPI2C1_Event_IRQHandler
259  .word     FMPI2C1_Error_IRQHandler
```

Peripherals Exceptions

Other Exception Handlers

- `startup_stm32f446xx.s` defines default handlers for all supported exceptions

```
294
295   .weak      WWDG_IRQHandler
296   .thumb_set WWDG_IRQHandler,Default_Handler
297
298   .weak      PVD_IRQHandler
299   .thumb_set PVD_IRQHandler,Default_Handler
```

- All defined as "**weak**", meaning that the `Default_Handler` (dead loop) can be overridden by custom handlers
- Custom handlers are all implemented in **`stm32f4xx_it.c`**
- HAL auto-generates empty custom handlers when an IRQ is enabled to avoid getting stuck in the `Default_handler` dead loop

In the beginning was the `Reset_Handler`

```
Reset_Handler:    /* In startup_stm32f446xx.s */
  ldr   sp, =_estack      /* set stack pointer */
  /* Copy the data segment initializers from flash to SRAM */
  b  LoopCopyDataInit /* calls subroutine LoopCopyDataInit */
  …
LoopCopyDataInit:
  bcc  CopyDataInit /* Copies .data section from ELF */
  …
  b  LoopFillZerobss /* Calls subroutine LoopFillZerobss */
LoopFillZerobss:
  bcc  FillZerobss /* Fills with zeros the .bss section (static vars) */
  bl  SystemInit  /* Calls the clock system intit func. in firmware */
  bl  main   /* Call the application's entry point.*/
  bx  lr
```

Called each time the MCU is reset (“boot”)

Extract of `stm32f4xx_it.c`

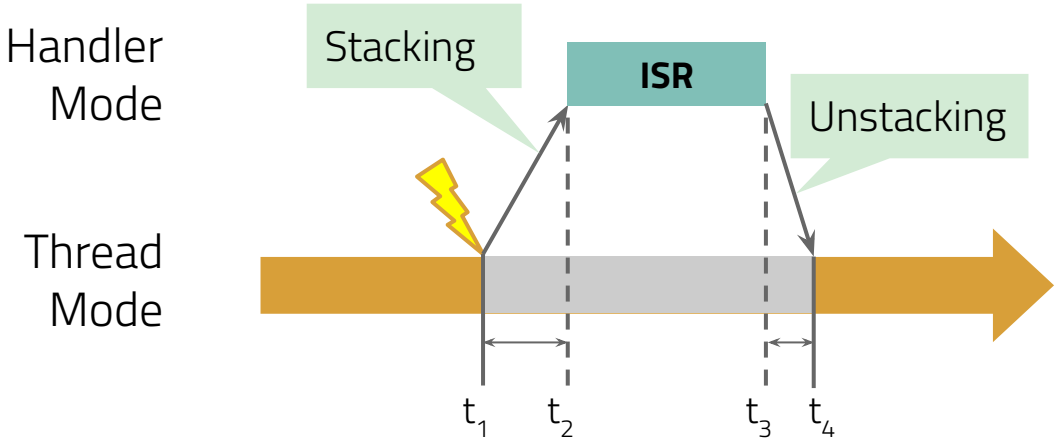
```
void HardFault_Handler(void) {
  /* USER CODE BEGIN HardFault_IRQn 0 */
  /* USER CODE END HardFault_IRQn 0 */
  while (1) {
    /* USER CODE BEGIN W1_HardFault_IRQn 0 */
    /* USER CODE END W1_HardFault_IRQn 0 */
  }
}
```

- The `HardFault_Handler` can be specialized here*, f.i., some registers could be "printed" before getting trapped in the `while(1)` dead loop.

(*) **Recall** put user code between the two BEGIN/END comments

How an ISR is called

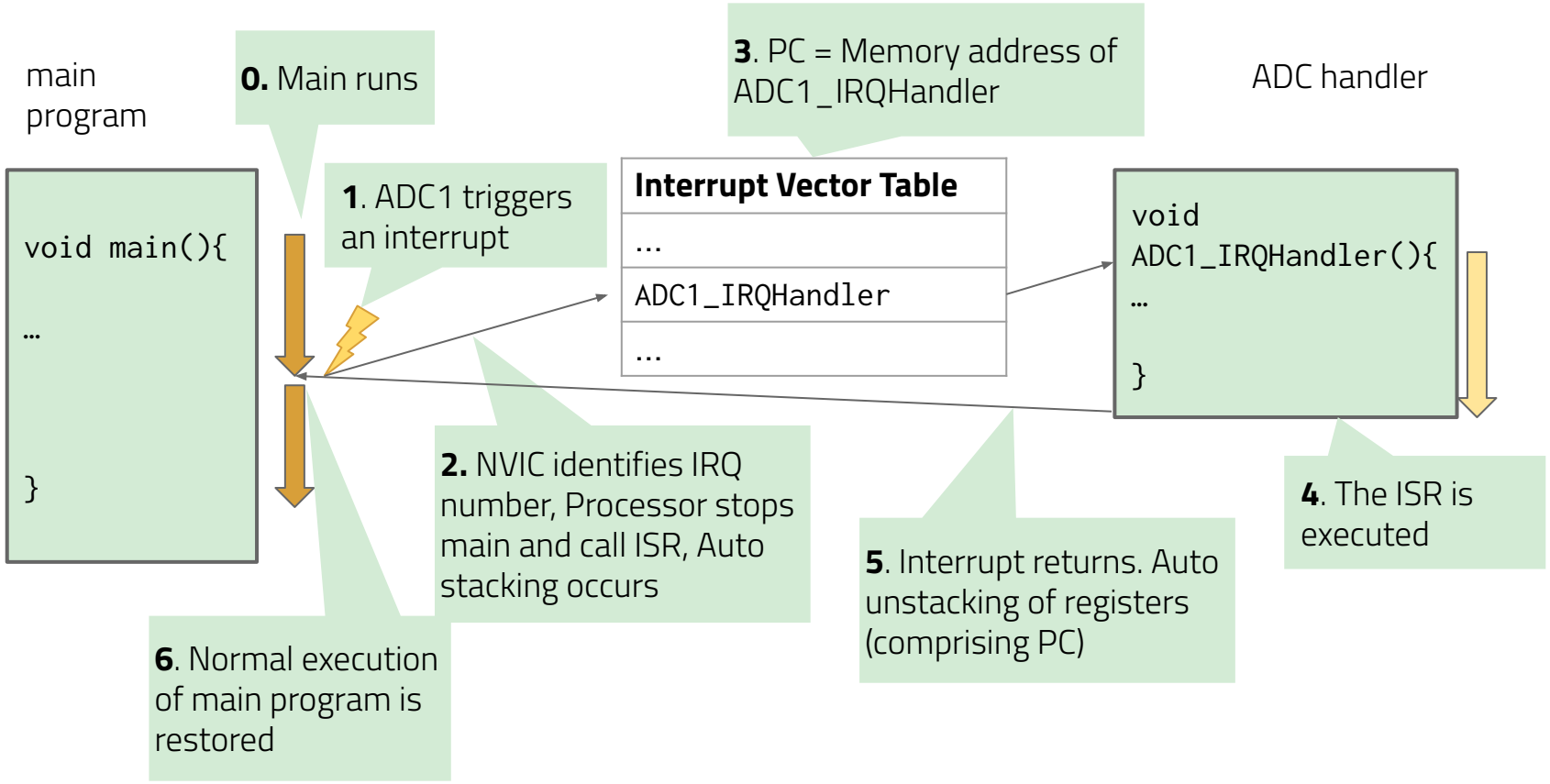
- **Interrupt Stacking**: before calling the ISR, 8 registers are automatically pushed on the stack to preserve the running environment (r0, …,r3, r12, LR, PSR, and **PC**)
- **Interrupt Unstacking**: after the ISR completes, the 8 registers are automatically popped off the stack recovering the environment



t_2-t_1: maximum 12 cycles*
t_4-t_3: 10 cycles*
Interrupt Latency = t_2-t_1 + t_4-t_3

*) larger if in FPU context!

Interrupt handling



How to handle exceptions programmatically

Exception status in ARM

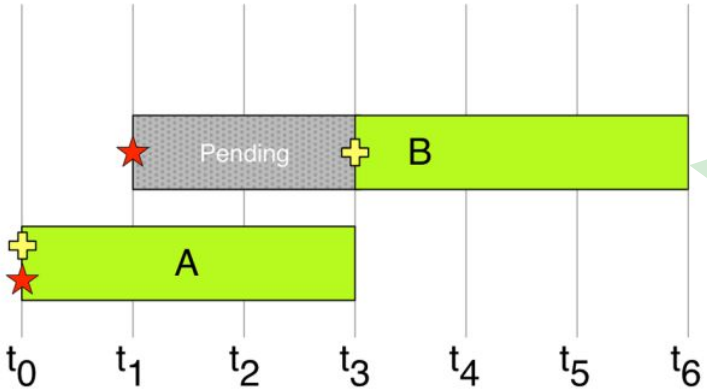
An exception can be:

- Disabled or enabled:
  - When disabled the processor will not be “notified” of that event (NMIs are always enabled!)
- Pending or not pending:
  - when “pending” the interrupt has been fired but it is waiting to be served
- Active or inactive:
  - when “active” it is being served (CPU is running ISR instructions)

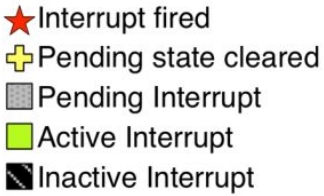
Exception handling lifecycle

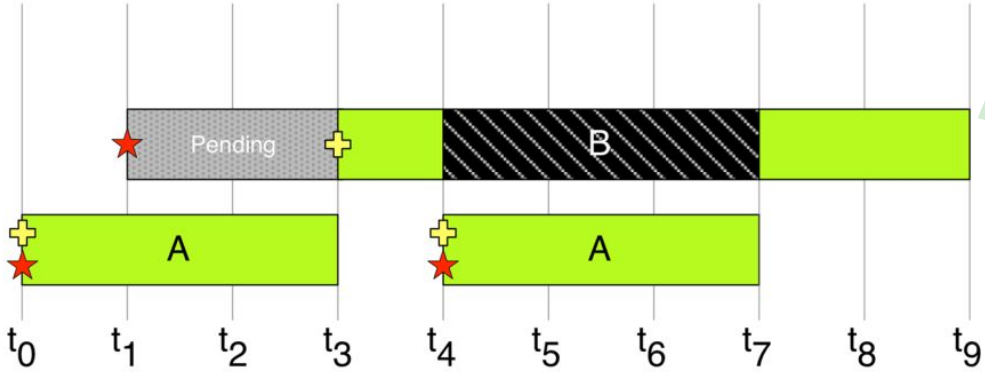
- When an exception fires (exception was enabled) it is automatically set by the processor in **pending** state
- If no other interrupts are executing (i.e., processor is in thread mode) the pending state is automatically cleared
- Now the processor starts serving the exception calling the ISR pointed by the interrupt vector
- IMPORTANT: the exception handler can be **preempted**, i.e. its execution can be put on "hold", if another exception with higher priority is fired!
- In this case the interrupted exception handler is put in "inactive state"

Pending exception & preemption examples



Interrupt B has a lower priority, it is pending until A finishes being served



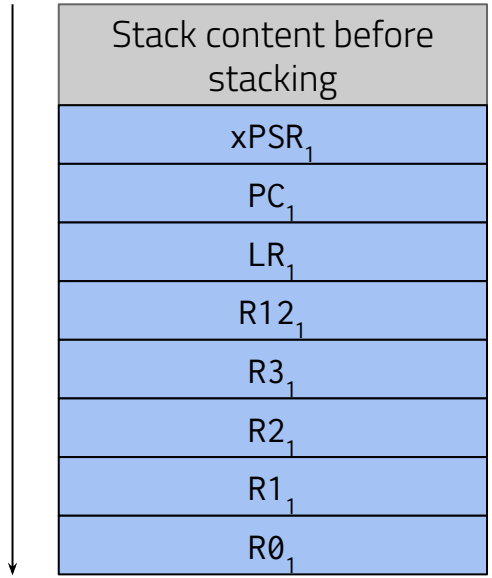


When A finishes being served, B starts to be served but gets preempted by A interrupt and put in “inactive” state. B resumes being served after A finishes

Example: stack content



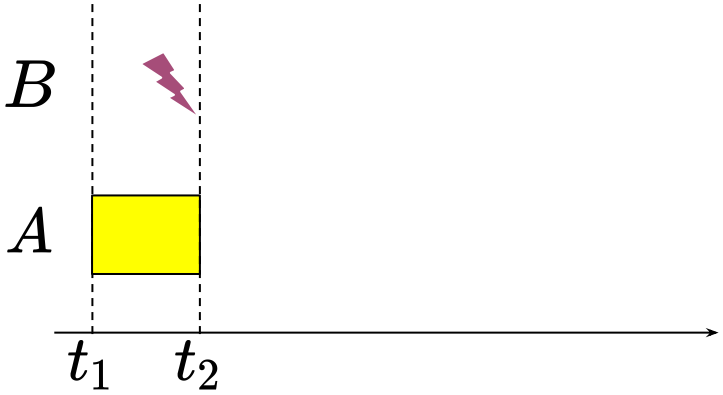
Exception A fires at t_1 registers are pushed to the stack and exception handler A is called

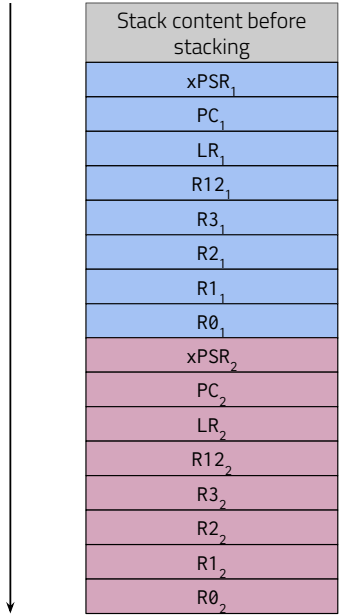


Thread mode

```
...
error = setpoint - measured_out;
u = Kp * (error + Ki * int_error + Kd * der_err);
return u;
```

Example: stack content



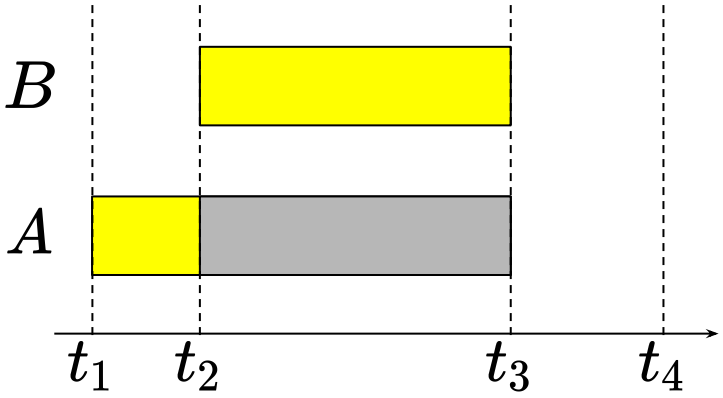


While in the A handler at t_2 B interrupt occur, registers are pushed to the stack and exception handler B is called

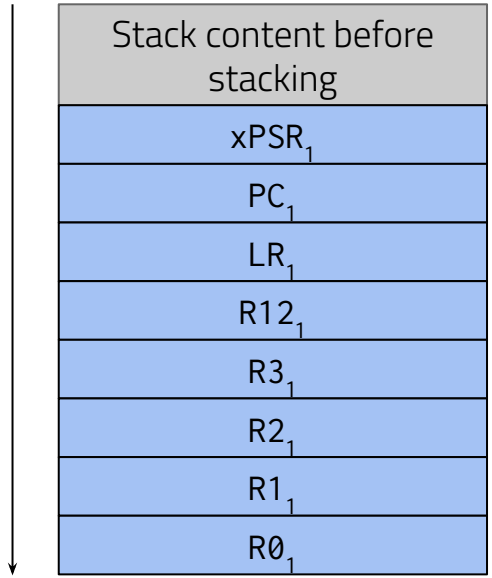
Handler mode: `A_Handler`

```
...
a_handle_flag = 1;
read_val = ...
```

Example: stack content



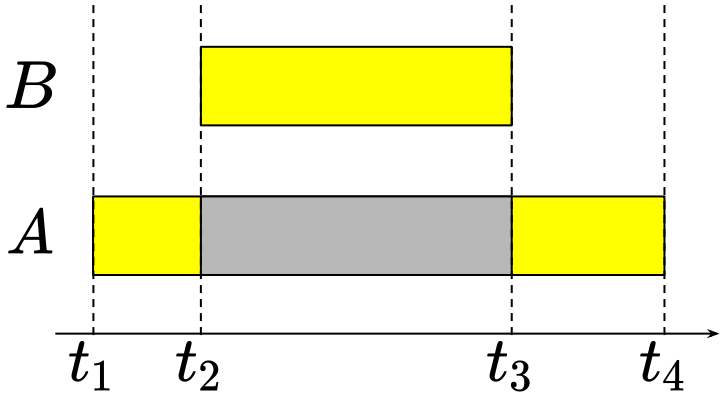
At time t_3 B handler returns, unstacking occurs which restores in the registers their content at time t_2 . A handler resumes.



Handler mode: `B_Handler`

```
...
some_var = 0x34;
return;
```

Example: stack content



At time t_4 A returns, unstacking occurs which restores in the registers their content at time t_1 right before the A handler was called. After t_4 thread mode is restored.

Stack content before stacking

Handler mode: `A_Handler`

```
a_handle_flag = 1;
read_val = ...
...
return;
```

Enabling/disabling interrupts in NVIC

- Cortex-M handles enabling/disabling interrupts via:
  - eight 32-bit **Interrupt Set-Enable Registers** (`ISER0 - ISER7`) . Used to **enable** a specific interrupt by setting the corresponding bit
  - eight 32-bit **Interrupt Clear-Enable Registers** (`ICER0-ICER7`). Used to **disable** a specific interrupt by setting the corresponding bit

CMSIS functions (IRQn is the IRQ number):

- `void NVIC_EnableIRQ(IRQn_Type IRQn)`
  `void NVIC_DisableIRQ(IRQn_Type IRQn)`

Interrupts number definitions

IRQ numbers are defined by the `IRQn_Type` enum in `stm32f446xx.h`

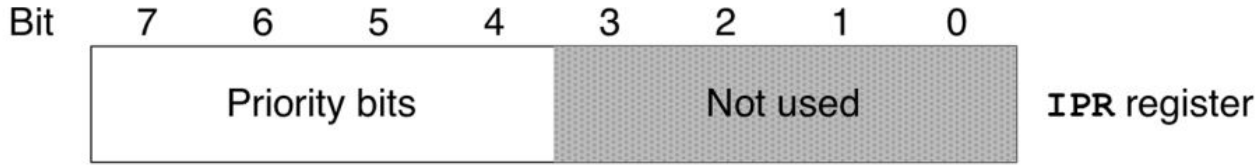
Cortex-M4 (independent of the silicon producer)

Periph IRQn dependent on the specific MCU

```
typedef enum
{
/******  Cortex-M4 Processor Exceptions Numbers ***********************************************************/
  NonMaskableInt_IRQn        = -14,    /*!< 2 Non Maskable Interrupt                                */
  MemoryManagement_IRQn      = -12,    /*!< 4 Cortex-M4 Memory Management Interrupt                 */
  BusFault_IRQn              = -11,    /*!< 5 Cortex-M4 Bus Fault Interrupt                         */
  UsageFault_IRQn            = -10,    /*!< 6 Cortex-M4 Usage Fault Interrupt                       */
  SVCall_IRQn                = -5,     /*!< 11 Cortex-M4 SV Call Interrupt                          */
  DebugMonitor_IRQn          = -4,     /*!< 12 Cortex-M4 Debug Monitor Interrupt                    */
  PendSV_IRQn                = -2,     /*!< 14 Cortex-M4 Pend SV Interrupt                          */
  SysTick_IRQn               = -1,     /*!< 15 Cortex-M4 System Tick Interrupt                      */
/******  STM32 specific Interrupt Numbers *****************************************************************/
  WWDG_IRQn                  = 0,      /*!< Window WatchDog Interrupt                               */
  PVD_IRQn                   = 1,      /*!< PVD through EXTI Line detection Interrupt               */
  TAMP_STAMP_IRQn            = 2,      /*!< Tamper and TimeStamp interrupts through the EXTI line   */
  RTC_WKUP_IRQn              = 3,      /*!< RTC Wakeup interrupt through the EXTI line              */
  FLASH_IRQn                 = 4,      /*!< FLASH global Interrupt                                  */
  RCC_IRQn                   = 5,      /*!< RCC global Interrupt                                    */
  EXTI0_IRQn                 = 6,      /*!< EXTI Line0 Interrupt                                    */
  EXTI1_IRQn                 = 7,      /*!< EXTI Line1 Interrupt                                    */
  EXTI2_IRQn                 = 8,      /*!< EXTI Line2 Interrupt                                    */
  EXTI3_IRQn                 = 9,      /*!< EXTI Line3 Interrupt                                    */
  EXTI4_IRQn                 = 10,     /*!< EXTI Line4 Interrupt                                    */
  DMA1_Stream0_IRQn          = 11,     /*!< DMA1 Stream 0 global Interrupt                          */
  DMA1_Stream1_IRQn          = 12,     /*!< DMA1 Stream 1 global Interrupt                          */
  DMA1_Stream2_IRQn          = 13,     /*!< DMA1 Stream 2 global Interrupt                          */
  DMA1_Stream3_IRQn          = 14,     /*!< DMA1 Stream 3 global Interrupt                          */
  DMA1_Stream4_IRQn          = 15,     /*!< DMA1 Stream 4 global Interrupt                          */
  DMA1_Stream5_IRQn          = 16,     /*!< DMA1 Stream 5 global Interrupt                          */
  DMA1_Stream6_IRQn          = 17,     /*!< DMA1 Stream 6 global Interrupt                          */
  ADC_IRQn                   = 18,     /*!< ADC1, ADC2 and ADC3 global Interrupts                   */
```

Exception Priorities

- Exceptions can be assigned with priorities to regulate the relative “importance” of events
- A higher urgency is assigned with a **lower priority number** (highest priority is 0)
- In Cortex-M3/4/7 each exception’s priority is defined using the 8-bit **IPR** register



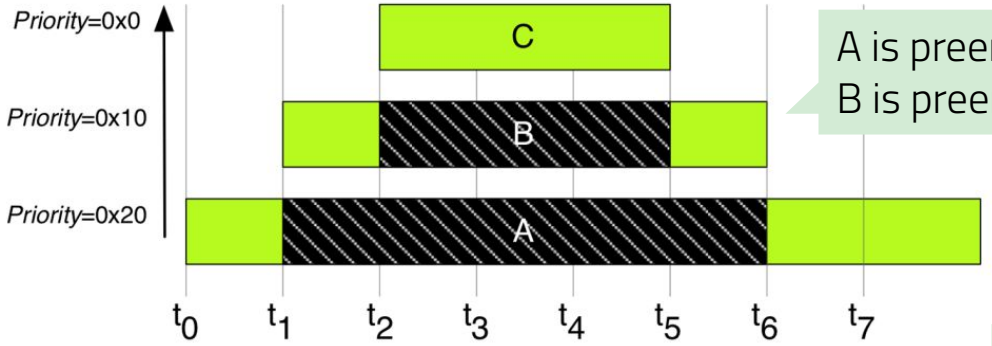
- Since only the 4 highest bits are used we have 16 levels of priority available (0x00, 0x10,0x30,...,0xF0)

Preemption priority and sub-priority

- Cortex-M3/4/7 allows to have a higher control of interrupts by further dividing the 4 priority bits into
  - preemption priority bits (higher bits)
  - sub-priority bits (lower bits)
- If an interrupt has a higher preemption priority (a lower number!) it will preempt an interrupt having a lower preemption priority
- The sub-priority bits define which ISR is executed first in case of multiple **pending** ISRs, but does not have any effect on preemption
- To set priority to IRQn with HAL use:

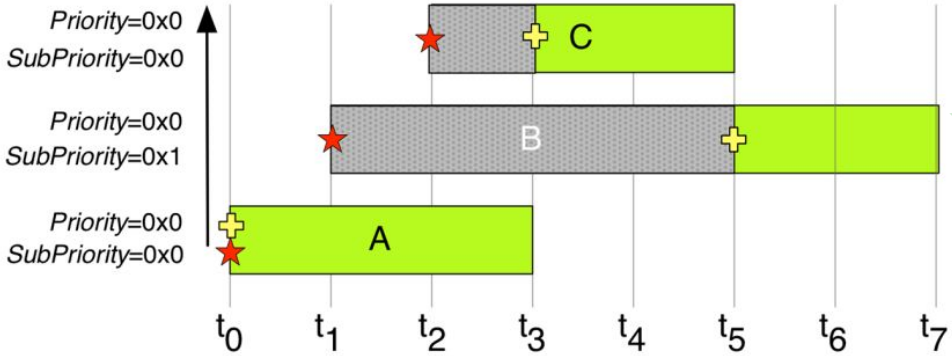
```
void HAL_NVIC_SetPriority(IRQn_Type IRQn,
                          uint32_t PreemptPrio, uint32_t SubPrio);
```

Examples: preemption, sub-priority



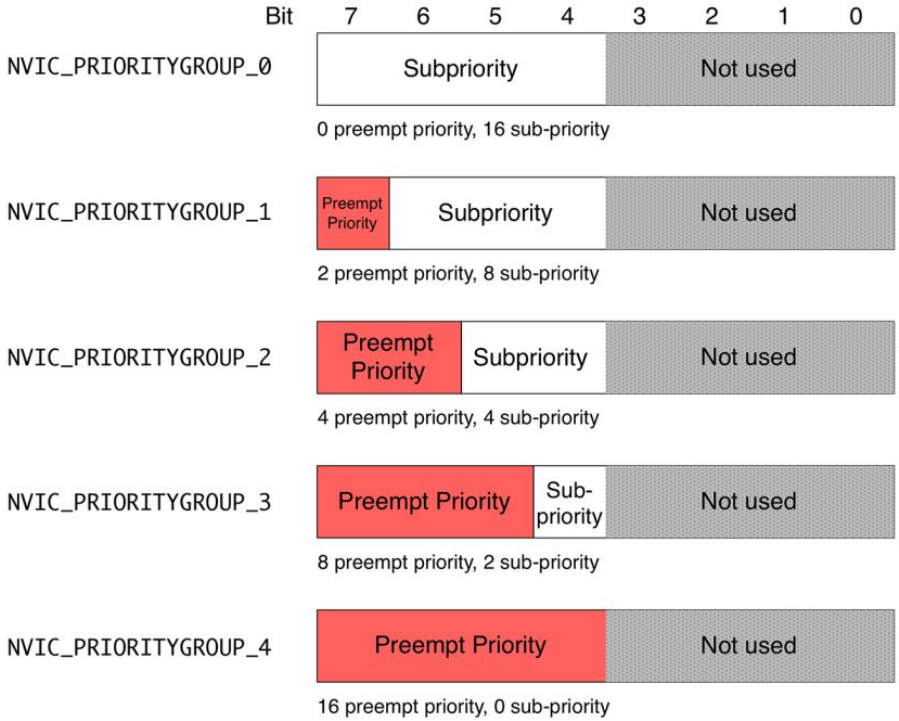
A is preempted by B
B is preempted by C





All interrupts same preemption priorities
When A finishes, C is the first being served since B had a lower sub-priority then C

All possible IPR modes



The way IPR is logically divided into **preemption priority** and **sub-priority** is defined in a register and has effect on all the exceptions!

To do that use:

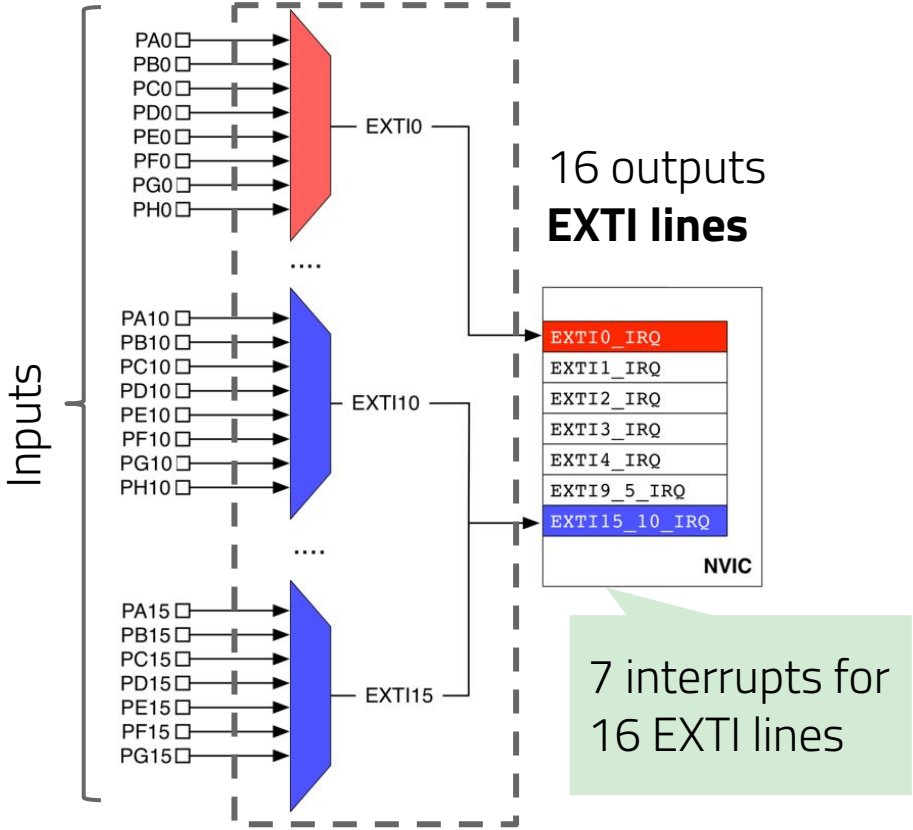
```
HAL_NVIC_SetPriorityGrouping(uint32_t PriorityGroup)
```

Example: using HAL to define an interrupt

```
HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP3);
HAL_NVIC_Enable_IRQ(ADC_IRQn);      /* Enable interrput */
HAL_NVIC_Enable_IRQ(USART1_IRQn);   /* Enable interrput */
HAL_NVIC_Enable_IRQ(USART2_IRQn);   /* Enable interrput */
HAL_NVIC_SetPriority(ADC_IRQn, 0x1, 0);     /* Set priority */
HAL_NVIC_SetPriority(USART1_IRQn, 0x2, 0);  /* Set priority */
HAL_NVIC_SetPriority(USART2_IRQn, 0x2, 1);  /* Set priority */
/* Q1. What happens if ADC_IRQn fires while USART2_IRQn
 *     is being served?
 * Q2. What happens if ADC_IRQn is being served and both
 *     USART1_IRQn and USART2_IRQn fire? How is served
 *     first and when?                                  */
```

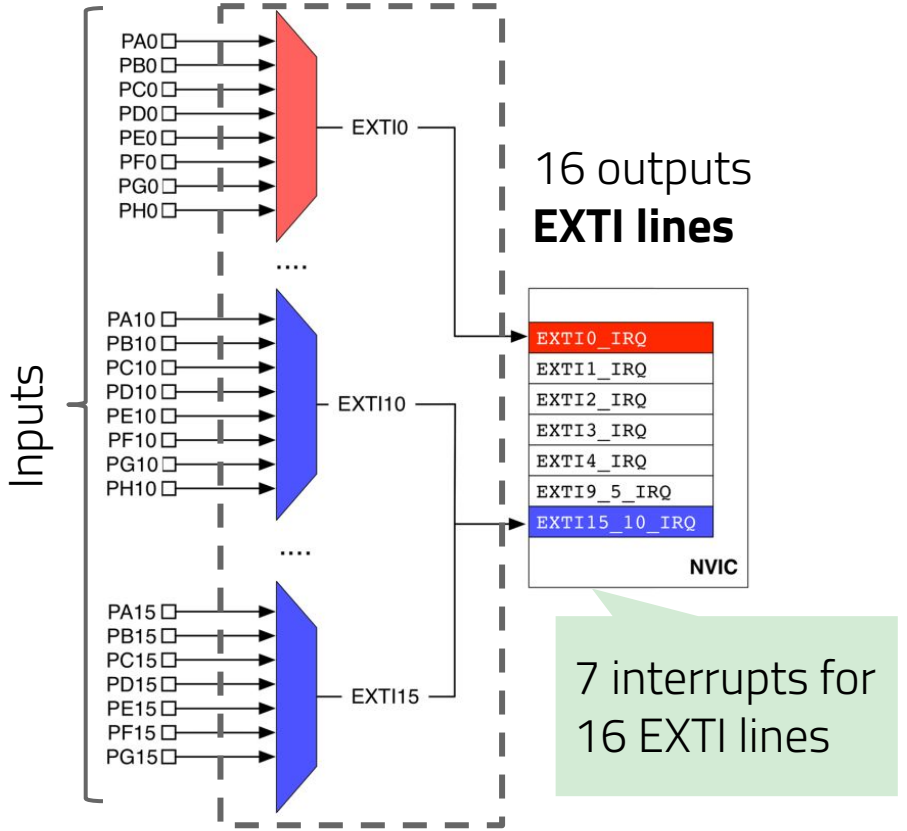
Handling external Interrupts: EXTI controller

- GPIOs are connected to EXTI lines (as shown)
- Several GPIOs share the same EXTI line (f.i., `PX0` share `EXTI0`)
- EXTI lines are associated to an IRQn (`EXTIX_IRQ`)
- Some EXTI lines are associated to the same IRQn (f.i., **`EXTI9_5_IRQ`**)



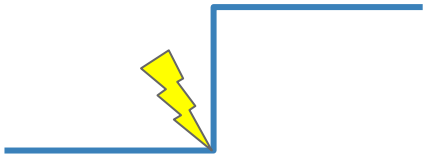
EXTI limitations

- Only one of the pins connected to an EXTI line can be used to generate an interrupt (f.i., you cannot generate interrupts from PA0 and PB0)
- EXTI lines sharing the same IRQ will need to programmatically discriminate the line that caused the interrupt

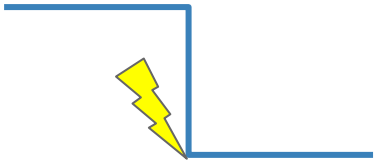


EXTI lines triggers

GPIO_MODE_IT_RISING



GPIO_MODE_IT_FALLING

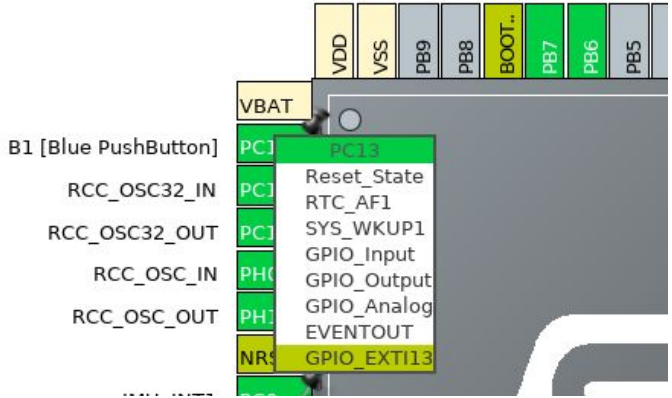


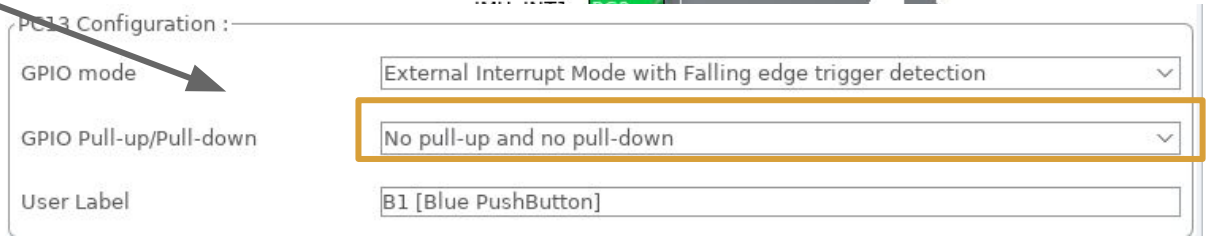
GPIO_MODE_IT_RISING_FALLING

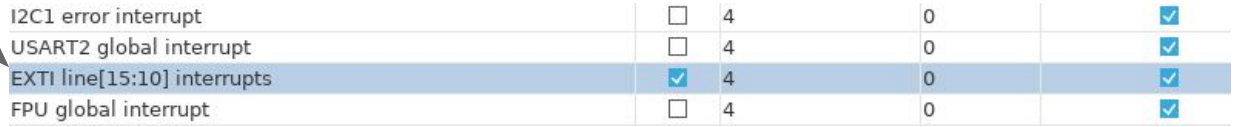


Example: button B1 generating an interrupt

- Set PC13 to **GPIO_EXTI13**
- Select the GPIO mode, f.i., External “Interrupt model with falling edge trigger detection”
- Enable the interrupt through NVIC config







Generated code

gpio.c

```
  GPIO_InitStruct.Pin = B1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
…
void EXTI15_10_IRQHandler(void) { /* stm32f446xx_it.h */
  /* USER CODE BEGIN EXTI15_10_IRQn 0 */
  /* USER CODE END EXTI15_10_IRQn 0 */
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13);
  /* USER CODE BEGIN EXTI15_10_IRQn 1 */
  /* USER CODE END EXTI15_10_IRQn 1 */
}
```

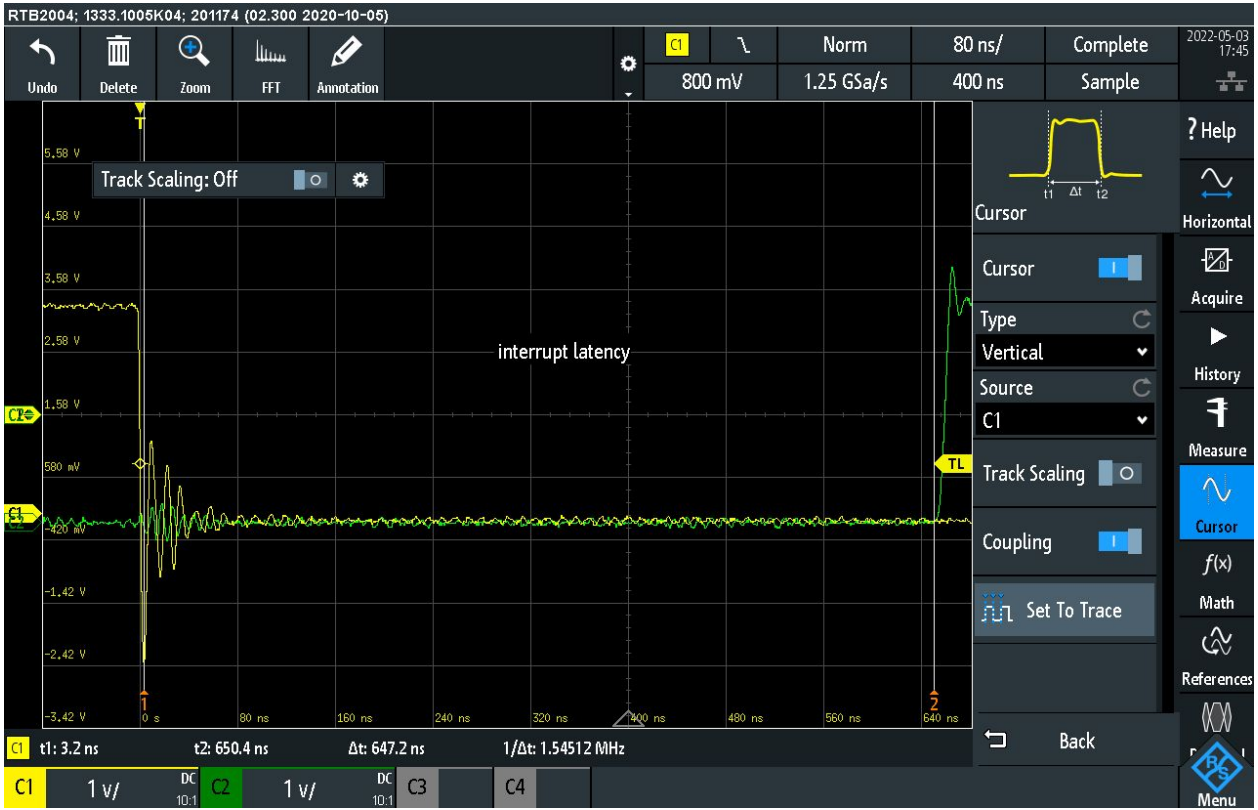
How IRQ is handled by default

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
  /* EXTI line interrupt detected */
  if(__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != RESET)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Callback(GPIO_Pin); /*user defined */
  }
}
```

User code goes in HAL_GPIO_EXTI_Callback

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == B1_Pin ) {
      /* Do something, f.i., turn on the LED! */
   }
}
```

Measuring the latency - interrupt



Latency: 647ns

Clock: 84 MHz

One clock period: ~12ns

Clock cycles: 647/12 ~= 53

(12 for stacking)

Measuring the latency - busy wait



Latency: 2.3 us

~4x compared to interrupt

Wasted Clock cycles:

2300/12~=191

Temporarily Disabling exceptions

- Sometimes we need a section of code not to be interrupted
- Such sections are typically called “critical sections”
- We will deal more specifically of this sections when we will talk about concurrency in multi-tasking RTOS
- To implement this we can temporarily disable (maskable) exceptions
- Three registers are involved:
  - BASEPRI: when non-zero, disables all exceptions with priority number higher or equal than BASEPRI
  - PRIMASK: if equal to 1, all interrupts disabled
  - FAULTMASK: if equal to 1, all exceptions disabled

Temporarily Disabling exceptions

- `__disable_irq()`: atomic CMSIS function (`CPSID i`) to temporarily disable all the IRQ interrupts (sets PRIMASK to 1)
- `__enable_irq()`: atomic CMSIS function (`CPSIE i`) to enable IRQ (sets PRIMASK to 0)
- **NOTE**: Disabling IRQ could compromise the way your system works, because you might miss some important interrupts
- If an interrupt condition occurs while interrupts are masked that request will remain **pending** until its ISR will be called
- In practice you should disable IRQ for a very short amount of time.

Critical sections should always be very short and fast!

Exercise: toggle LD2 when B1 is pressed

- Toggle the LED LD2 each time the button B1 is pressed
- Implement the code to serve the interrupt in the callback **`HAL_GPIO_EXTI_Callback`**
- Experiment with both `GPIO_MODE_IT_RISING` and `GPIO_MODE_IT_FALLING`
- See what happens if you set the interrupt in rising mode when you hold B1 a while and then release it
- Put a breakpoint at the beginning of the ISR **`EXTI15_10_IRQHandler`** and do a step-by-step analysis of the program flow in response to the IRQ